

XSS Discovery

By now, we should have a good understanding of what an XSS vulnerability is, the three types of XSS, and others. We should also understand how XSS works through injecting JavaScript code into the client-side page, and additional code, which we will later learn how to utilize to our advantage.

In this section, we will go through various ways of detecting XSS vulnerabilities within a web application. For (and all vulnerabilities in general), detecting them can become as difficult as exploiting them. However, as XSS is widespread, many tools can help us in detecting and identifying them.

Automated Discovery

Almost all Web Application Vulnerability Scanners (like [Nessus](#), [Burp Pro](#), or [ZAP](#)) have various capabilities to detect XSS vulnerabilities. These scanners usually do two types of scanning: A Passive Scan, which reviews client-side based vulnerabilities, and an Active Scan, which sends various types of payloads to attempt to trigger an XSS on the page source.

While paid tools usually have a higher level of accuracy in detecting XSS vulnerabilities (especially when scanning the page source), we can still find open-source tools that can assist us in identifying potential XSS vulnerabilities. Such tools usually inject payloads into input fields in web pages, sending various types of XSS payloads, and then comparing the rendered page source to see if the payload can be found in it, which may indicate a successful XSS injection. Still, this will not always be accurate. If the same payload was injected, it might not lead to a successful execution due to various reasons, so we must be careful with XSS injection.

Some of the common open-source tools that can assist us in XSS discovery are [XSS Strike](#), [Brute XSS](#), and [XSSer](#). We can install them by cloning it to our VM with `git clone`:

XSS Discovery

```
dbcyph0n@htb[/htb]$ git clone https://github.com/s0md3v/XSSStrike.git
dbcyph0n@htb[/htb]$ cd XSSStrike
dbcyph0n@htb[/htb]$ pip install -r requirements.txt
```

```
-----  
[+] Payload: <HtMl%09onPoIntERENTER+==+confirm(>  
[!] Efficiency: 100  
[!] Confidence: 10  
[?] Would you like to continue scanning? [y/N]
```

As we can see, the tool identified the parameter as vulnerable to XSS from the first payload. Try to verify this by testing it on one of the previous exercises. You may also try testing out the other tools on the previous exercises to see how capable they are in detecting XSS vulnerabilities.

Manual Discovery

When it comes to manual XSS discovery, the difficulty of finding the XSS vulnerability depends on the level of the application. Basic XSS vulnerabilities can usually be found through testing various XSS payloads, but identifying advanced vulnerabilities requires advanced code review skills.

XSS Payloads

The most basic method of looking for XSS vulnerabilities is manually testing various XSS payloads against a web application page. We can find huge lists of XSS payloads online, like the one on [PayloadAllTheThings](#) or the one in [Payloads](#). We can test these payloads one by one by copying each one and adding it in our form, and seeing whether an error message is displayed.

Note: XSS can be injected into any input in the HTML page, which is not exclusive to HTML input fields, but also to headers like the Cookie or User-Agent (i.e., when their values are displayed on the page).

You will notice that the majority of the above payloads do not work with our example web applications, even though they are the most basic type of XSS vulnerabilities. This is because these payloads are written for a wide variety of contexts (like after a single quote) or are designed to evade certain security measures (like sanitization filters). Furthermore, there are a variety of injection vectors to execute JavaScript code, like basic `<script>` tags, other `HTML Attributes` like `onmouseover` or `onload` attributes. This is why we can expect that many of these payloads will not work in all test cases, as they are not designed for all types of injections.

This is why it is not very efficient to resort to manually copying/pasting XSS payloads, as even if a web application is vulnerable, it may take us a while to identify the vulnerability, especially if we have many input fields to test. This is why it may be useful to write a Python script to automate sending these payloads and then comparing the page source to see how our payload was rendered.

vulnerabilities before release. For such cases, manual code review may reveal undetected XSS vulnerabilities in releases of common web applications. These are also advanced techniques that are out of the scope of this article. If you are interested in learning them, the [Secure Coding 101: JavaScript](#) and the [Whitebox Pentesting 101: Command Injection](#) cover this topic.



Connect to Pwnbox

Your own web-based Parrot Linux instance to play our labs.

Pwnbox Location

AU

Terminate Pwnbox to switch location

Start Instance

∞ / 1 spawns left

+ 2 

What type of XSS was found on the above server? "name only"

Submit your answer here...

+10 Streak pts

 Submit PreviousNext  Cheat Sheet Go to Questions

Table of Contents

XSS Basics

Intro to XSS

 Stored XSS Reflected XSS DOM XSS [XSS Discovery](#)

XSS Attacks

Defacing

 Phishing Session Hijacking

XSS Prevention

XSS Prevention

Skills Assessment

 Skills Assessment